

Dokumentasi Infrastruktur MinIO KOPIPU Smart

Versi: 1.0

Status: Production Ready

Tanggal Update Terakhir: 02/Agustus/2026 17:44 WIB

1. Latar Belakang dan Tujuan

Sebelumnya, aplikasi Laravel menyimpan file secara lokal di dalam direktori `storage/app/public`. Pendekatan ini memiliki beberapa kelemahan arsitektural:

- File terisolasi di satu server spesifik.
- Sulit untuk dipindahkan (migrasi) atau dicadangkan (backup).
- Tidak mendukung skalabilitas horizontal (sulit digunakan jika aplikasi di-deploy di banyak server/Load Balancer).
- Risiko kehilangan data tinggi (Single Point of Failure) jika server utama rusak.

Solusi: Membangun *Object Storage* mandiri menggunakan **MinIO** yang *S3-compatible*. Solusi ini memastikan file aman, terpusat, mudah di-backup, dan siap untuk diakses oleh berbagai platform (Web, Mobile App) secara bersamaan.

2. Topologi dan Arsitektur

Infrastruktur ini memisahkan jalur akses antara sistem (API) dan administrator (Dashboard) melalui Reverse Proxy Nginx.



Jalur Akses Dashboard Administrator:

Administrator → `https://minio.kopipu-smart.labkesdasumenep.cloud` → Nginx →
`localhost:9001` → MinIO Console

3. Komponen Server MinIO

Berikut adalah detail komponen spesifik yang berjalan di dalam VPS:

A. MinIO Server Application

Aplikasi utama yang bertugas menyimpan, menerima upload, mengirim, dan menghapus file.

- **Lokasi Binary:** `/usr/local/bin/minio`

B. Systemd Service

Dikonfigurasi agar MinIO berjalan otomatis di latar belakang (background) dan otomatis *restart* jika VPS mengalami *reboot*.

- **Nama Service:** `minio.service`
- **Perintah Manajemen:**
 - Cek Status: `systemctl status minio`
 - Menjalankan: `systemctl start minio`
 - Restart: `systemctl restart minio`
 - Stop: `systemctl stop minio`

C. Direktori Data

Tempat penyimpanan fisik (harddisk) dari semua file bucket MinIO.

- **Lokasi Folder:** `/var/lib/minio`

D. File Konfigurasi

Menyimpan variabel environment (*Root User*, *Password*, *Port*, *URL*).

- **Lokasi File:** `/etc/minio/minio.conf`

4. Manajemen Domain dan Jaringan

Untuk memisahkan trafik *public* (aplikasi) dan *private* (administrator), disiapkan dua domain berbeda.

Fungsi	Domain	Pengguna	Keterangan
API (S3)	storage.kopipu-smart.labkesdasumenep.cloud	Laravel & Mobile App	Digunakan secara <i>backend</i> . Tidak diakses manusia.
Dashboard	minio.kopipu-smart.labkesdasumenep.cloud	Administrator	GUI untuk manajemen MinIO (Login admin).

HTTPS & Reverse Proxy

- **Web Server:** Nginx (sebagai *Reverse Proxy*)
 - storage... meneruskan trafik ke localhost:9000
 - minio... meneruskan trafik ke localhost:9001
- **SSL/TLS:** Menggunakan sertifikat gratis dari **Let's Encrypt**. Koneksi dari Laravel ke MinIO sepenuhnya terenkripsi (HTTPS).

5. Struktur Bucket

Bucket adalah *container* utama penyimpanan file (seperti *virtual harddisk*).

- **Nama Bucket Utama:** kopipu

Struktur folder di dalam bucket akan dihasilkan secara dinamis oleh Laravel saat proses *upload*:
kopipu/

```

|— pasien/
|— kader/
|— hasil-lab/
|— export/
|— article/
|— logo/
|— profile/
|— temp/
|— backup/
|— archive/

```

6. Manajemen Akses & Keamanan (IAM)

Ini adalah lapisan keamanan paling krusial. Infrastruktur ini menerapkan prinsip *Least Privilege* dengan memisahkan akun Administrator dan Akun Aplikasi.

A. Root Account (Administrator)

- **Username:** kopipu-storage (Berasal dari env MINIO_ROOT_USER)
- **Password:** ***** (Berasal dari env MINIO_ROOT_PASSWORD)
- **Hak Akses:** ☒ **Penuh (Superadmin)**. Boleh membuat/menghapus bucket, mengelola user/policy, dan mengubah konfigurasi sistem.
- **Aturan:** ☒ **DILARANG KERAS** digunakan oleh Laravel/Aplikasi.

B. IAM User (Khusus Aplikasi Laravel)

- **Username:** kopipu-laravel
- **Password:** *****
- **Metode Pembuatan:** Melalui MinIO Client (mc admin user add...)
- **Tujuan:** Mengamankan infrastruktur. Jika server Laravel diretas, penyerang hanya memiliki akses ke file, namun **tidak bisa** menghapus bucket secara keseluruhan atau masuk ke dashboard admin.

C. Access Policy (Izin Akses)

Policy (aturan izin akses) dibuat untuk membatasi kopipu-laravel.

- **Nama Policy:** kopipu-policy
- **Izin Diberikan (✓):** Upload, Download, Delete file, List Bucket.
- **Izin Ditolak (✗):** Membuat/menghapus bucket, membuat user, mengubah policy, login dashboard.

Alur Otorisasi:

Policy: kopipu-policy → diterapkan ke → User: kopipu-laravel → untuk akses ke → Bucket: kopipu

7. Administrasi Terminal (MinIO Client / mc)

mc adalah CLI tool MinIO, bekerja layaknya php artisan di Laravel.

Dua profil (alias) telah dikonfigurasi di server:

1. **kopipu** : Menunjuk ke server MinIO dengan akses *Root/Admin*. Digunakan untuk konfigurasi tingkat lanjut (membuat user, policy, backup).
2. **kopipu-laravel** : Menunjuk ke server MinIO dengan akun Laravel. Digunakan secara spesifik untuk **pengujian** (testing) pembatasan hak akses.

8. Prosedur Pengujian Infrastruktur

Pengujian dilakukan menggunakan alias kopipu-laravel untuk memvalidasi integrasi:

1. **Uji Upload:**
mc cp test.txt kopipu-laravel/kopipu/temp/

(Ekspektasi: Berhasil)

2. **Uji Lihat Daftar File:**
mc ls kopipu-laravel/kopipu/temp

(Ekspektasi: Muncul file test.txt)

Kesimpulan Pengujian: User benar, Policy terpasang tepat, Bucket aktif, koneksi HTTPS valid. MinIO siap di-consume oleh Laravel.

9. Integrasi Aplikasi (Laravel)

Agar Laravel dapat menggunakan infrastruktur ini, konfigurasi .env pada aplikasi Laravel harus diatur sebagai berikut:

```
FILESYSTEM_DISK=s3
AWS_ACCESS_KEY_ID=kopipu-laravel
AWS_SECRET_ACCESS_KEY=*****
AWS_DEFAULT_REGION=us-east-1
AWS_BUCKET=kopipu
AWS_ENDPOINT=https://storage.kopipu-smart.labkesdasumenep.cloud
```

AWS_USE_PATH_STYLE_ENDPOINT=true

Dengan konfigurasi di atas, developer cukup memanggil `Storage::disk('s3')` di kodingan Laravel tanpa perlu merubah logika bisnis aplikasi.

10. Ringkasan Keseluruhan

Infrastruktur ini bukan sekadar "tempat upload file", melainkan layanan *Object Storage* level produksi yang mengutamakan keamanan dan skalabilitas. Pemisahan yang tegas telah dilakukan pada lapisan:

1. **Infrastruktur** (MinIO Server S3-compatible).
2. **Akses** (Trafik API terpisah dari Dashboard Web).
3. **Keamanan** (Role Root Administrator vs Role IAM Application).
4. **Penyimpanan** (Bucket khusus dengan folder dinamis).

